Final Project Design Phase

Computer Science - Week 11

Oct 3, 2026 - Version 0.0.4

For complete instructions and project ideas, refer to the assignment in Canvas. Specifically, review the final project requirements before you continue below. If you have not reviewed the project requirements in Canvas (you might have to go to the modules view and check the current or past week's content), you will not be able to properly complete the parts below. In addition, it may be helpful to review the information on this Drafter documentation page.

0) Project Planning and Preparation

Successful programmers spend time planning before they begin coding. *After* reviewing the project requirements, answer the following questions to help guide your design and development process.

A. What concepts from class or previous assignments might help you complete this project? Which programming concepts (loops, conditionals, lists, classes, etc.) do you expect to use? What programming skills do you want to improve while completing this project?

I think implementing nested elements (lists inside of lists, lists inside of dataclasses) will be essential for my project, especially for the data storage to make it work (by mutating State or nested elements in State). The utilization of truthiness or falsiness to determine whether a function executes will also be essential. Functions that call other helper functions will also be useful for maintaining the website's readability. I will have to use loops, lists, dataclasses, and if statements. I don't expect to be doing most of the actual programming because that will be the LLM's job. Therefore, I expect to improve my ability to organize entire programs for readability while still maintaining functionality.

B. Which parts of the project do you expect to be the easiest for you? The hardest? Why?

I expect the actual coding itself to be the easiest part. I have some rough metrics gathered from other LLM-heavy projects to compare the estimated time to build my website by: a simple message board with networking takes one afternoon, implementing the AI logic for NPC character that dynamically reacts to the world takes maybe 1 day at most, and a full-on mod of a game from scratch, with zero human

intervention beyond the initial prompt, took roughly 2 weeks. Given these figures, I expect my project to take maybe 4 days max. The hardest part will be ensuring that everything works as it's supposed to. The LLM could implement code that works for a bit but is buggy, or code that seems to work but doesn't actually do what I want it to do. I also expect balancing the player progression in the game to take a long time.

C. Even though you have not yet designed the final project, estimate which tasks will likely require the most time and effort? Which features or components should be implemented first?

Design and bug fixing will require the most time and effort. The closer something is to a "game" that takes more complex user interactions rather than some sort of software or tool, the more bugs will pop up per piece of code. I need to come up with a basic version of the website that checks all the requirements, so I have something to hand in and implement that first. Then, I can add more things to the website if time allows.

D. What distractions might interfere with your work and how will you minimize them? What resources will you need available while working? What questions, if any, do you still have about the assignment requirements?

Some distractions that might interfere with my work include doomscrolling and other work I have to do. I will minimize doomscrolling by finding a way to make myself invested in my project. I will minimize other work I have to do by getting most of the website done in the next 2 weeks, when there are limited/few exams.

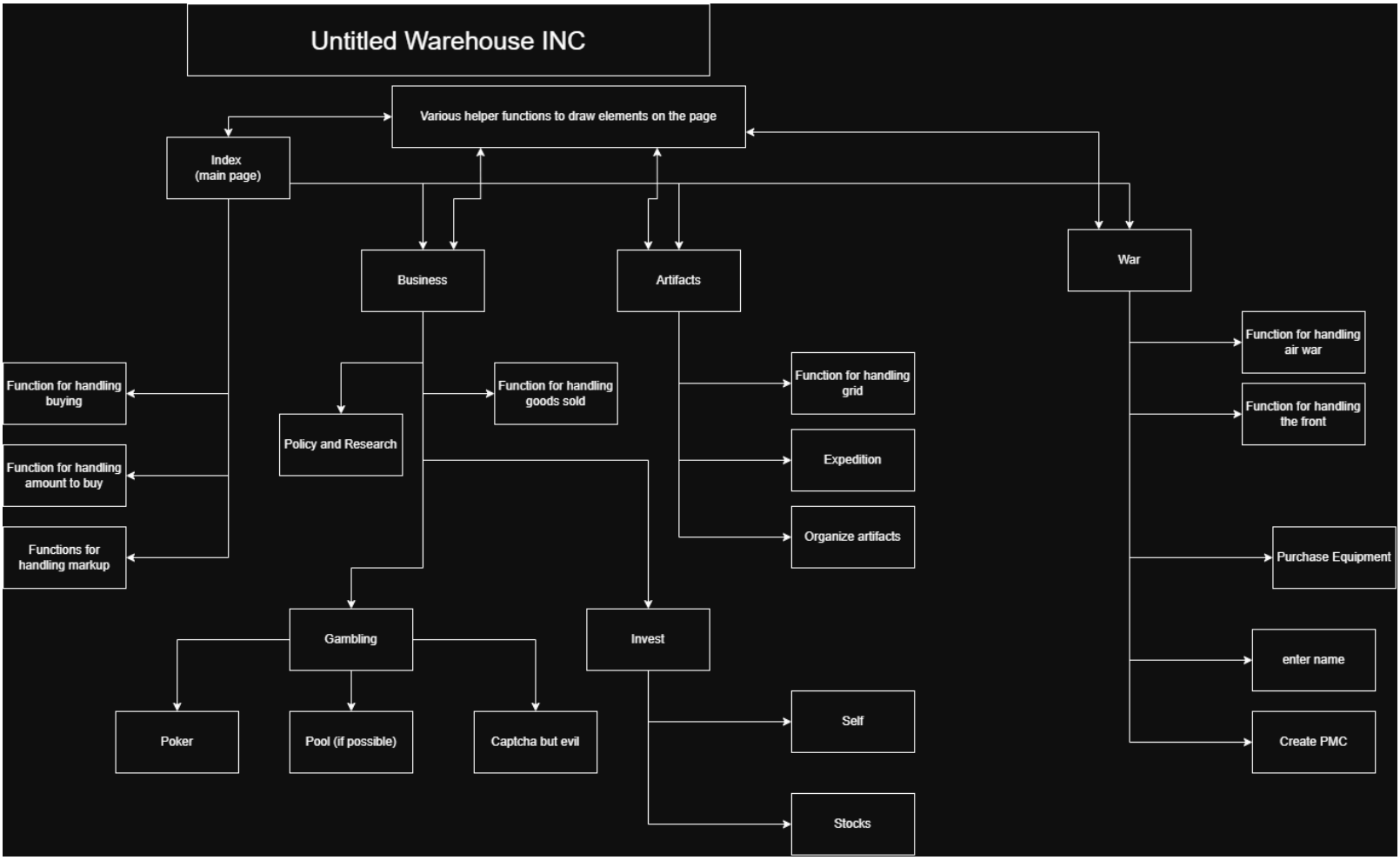
1) Site Diagram

Create a mock-up of your planned website. We suggest you make the mockup on paper, although you can also use software like PowerPoint or Slides to make a mockup. There is other design software that could also be used, but we strongly encourage you to keep it simple. Do not spend more than 15-30 minutes detailing the mock-up. Your goal should be to make a very quick and dirty diagram, to show that you have some kind of idea.

We recommend drawing each page as a rectangle, and laying out the text, buttons, and other components inside. Then, use arrows to indicate links between pages. Label each page the way you will name each function, and then put labels on the arrows to indicate what the button links. Also write down any parameters that should come into the pages.

In addition to the pages and routes, write down what fields your `State` will have, both their names and types. Think clearly about what data will need to be persistent across the entire website.

Add photo(s) or screenshot(s) of your website mock-up and labeled diagram on the next page



.

At minimun

```
@dataclass
class Progression:
    has_unlocked_fordov_van: bool
@dataclass
class CompanyAttributes:
    admin: int
    input_level: int
    output_level: int
    customer_loyalty: int
    goods_bonus: float
@dataclass
class Artifact:
    has_obtained:bool
    goods_bonus:int
    admin_bonus:int
… and so on
```

State:

```
buy_amount: int
goods_stocked_per_second: float
goods_stocked: int
markup: float
world_news: str
war_score: int
progression: Progression
obtained_artfacts: list[Artifact]
master_artifacts: list[Artifact]
company_attributes: CompanyAttributes
… and so on
```

index page:

For the next pages:

The buttons at the top call the route functions of the website that return pages

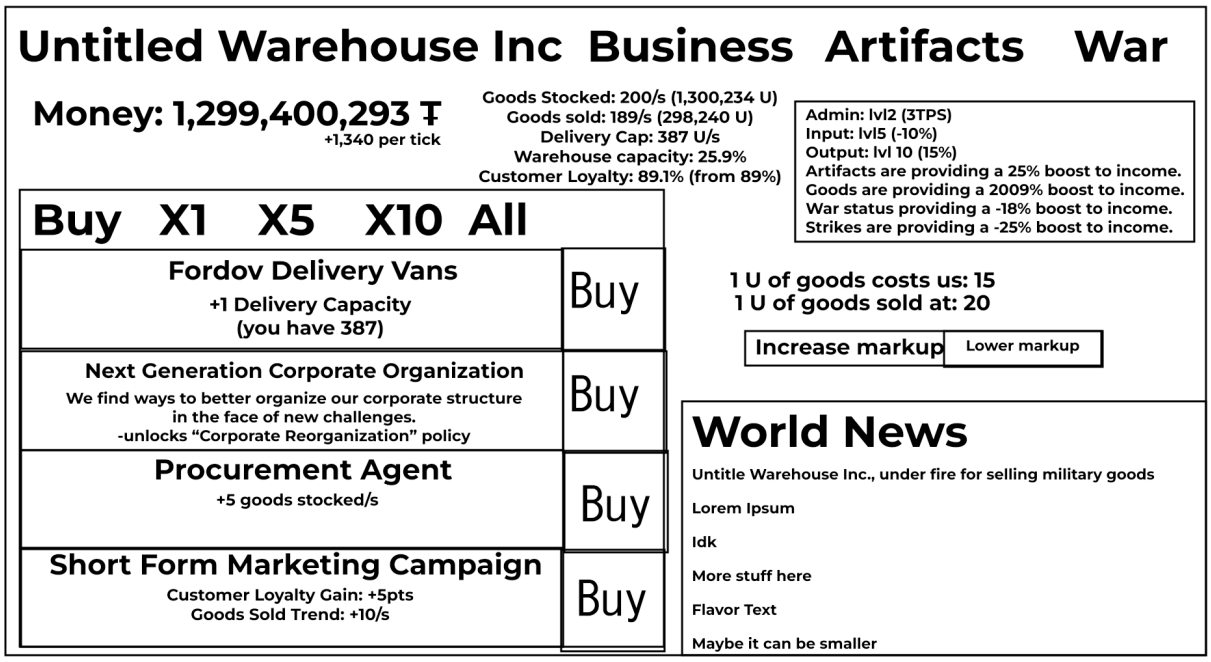
X1, X5, X10 buttons will call their own X##_adjust() function to change a parameter in state called buy_amount

handle_purchase_option() creates the list of things to buy with their associated buy buttons. State will have parameters goods_stocked_per_second, goods_stocked, etc which handle_purchase_option () will modify. I'm not sure how handle_purchase_option will get the list of things to show, maybe with a dataclass in state called Progression with parameters like "**has_unlocked_fordov_van: True**" and similar and a massive list of if statements in handle_purchase_option() to check. I will have to run this by an LLM later.

World News could be a string modified by functions while the game runs and which index just displays.

Attributes like admin, input, output, customer loyalty are stored in State, maybe under a dataclass like Company_Attributes for neatness.

Increase markup and lower markup will call their own **increase_markup() and lower_markup()** functionsand change a parameter in State called markup



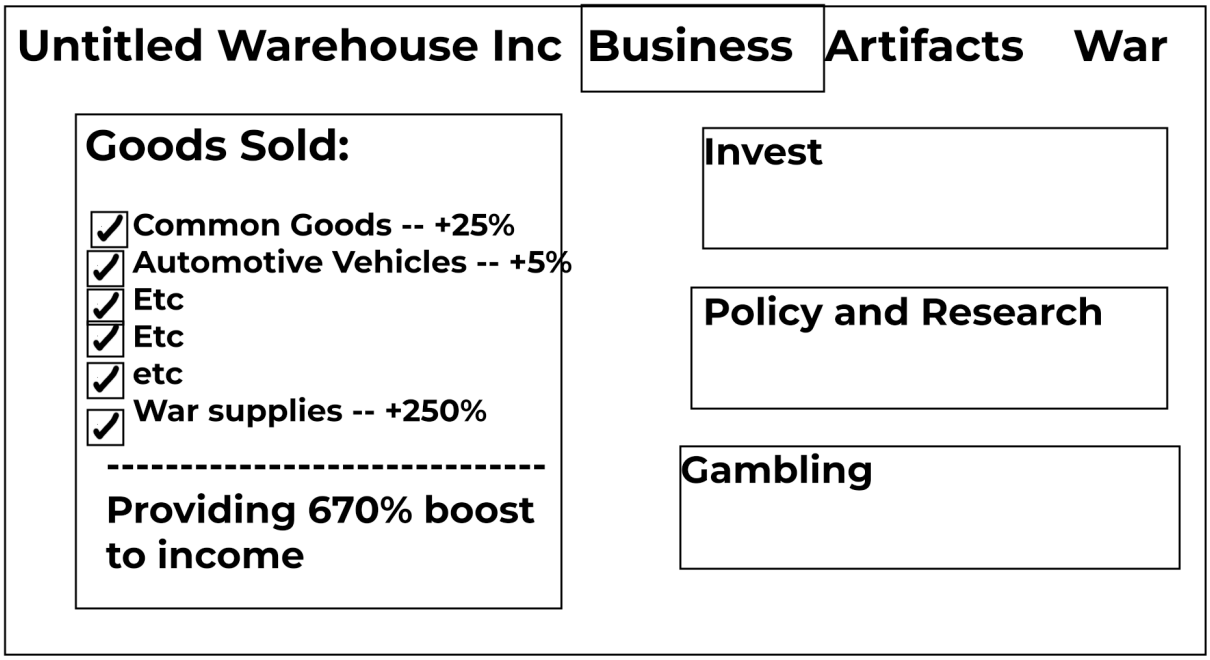
business page:

Incoming boost from goods is dealt with by handle_goods_sold() which draws the goods sold panel. Based on which goods are checked, it will update a parameter in State or State.company (company parameter is of type Company_Attributes) called goods_bonus.

Invest button calls the invest_page route function, which returns a page. A list of Stock dataclass inside State will be tracked for the stock section. Parameters in a Company_Attribute class in State will be used to track things like Admin lvl, output lvl, input lvl (which are things the player can invest in).

Policy and research is handled similarly. A dataclass Policy inside State will track the policies the player has active. A function like handle_policy_options() will handle what policy to show to the player. Research is tracked by a Research dataclass in State and changed by handle_research() and handle_research_options().

At the bare minimum, gambling just recreates the code from project 1, but you work with in-game money instead of some arbitrary points.



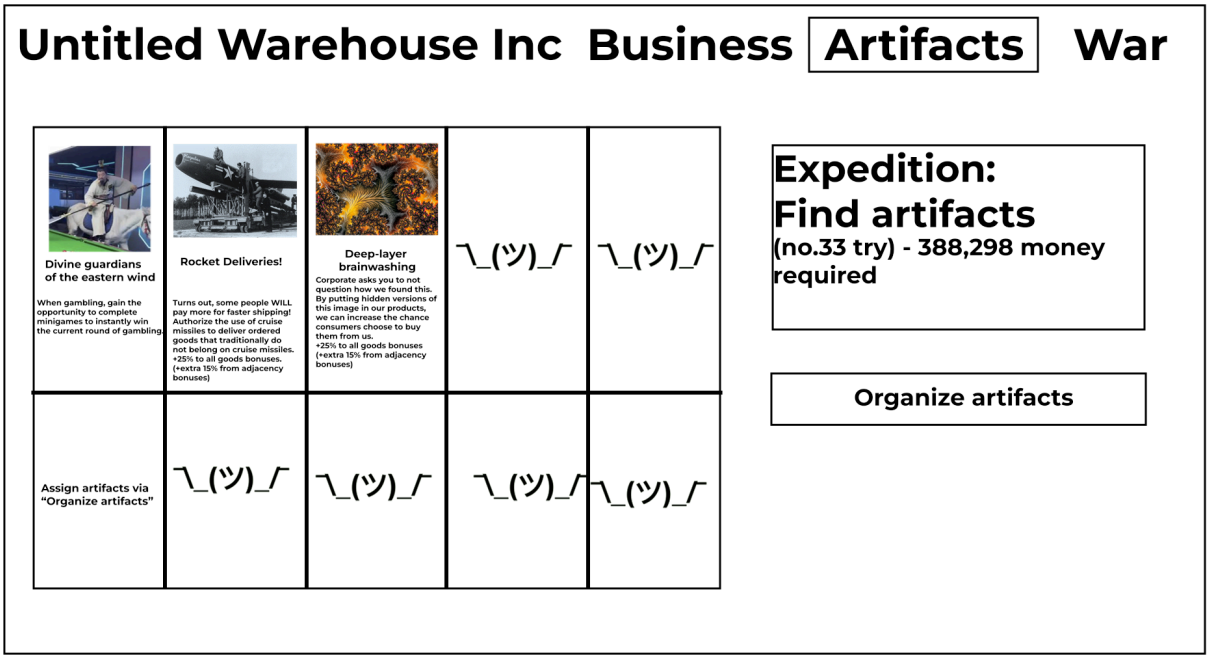
Artifacts page:

A function draw_artifact_grid() creates the grid of artifacts for the player to see. It will probably also have to call something like get_adjacency_bonus(). This requires tracking artifacts as a list of Artifacts dataclass.

Somewhere else there will also have to be a master list of artifacts created at website start.

find_artifacts() will take an artifact and, based on a rarity value (which may be a parameter of Artifact datalcass), mark one as obtained by the player, at which point something else adds it to the list of Artifacts in State that represents the ones the player obtains.

I do not know how organize_artifacts() will even begin to work, but it needs to let the player change what artifacts are on the grid.



War page:

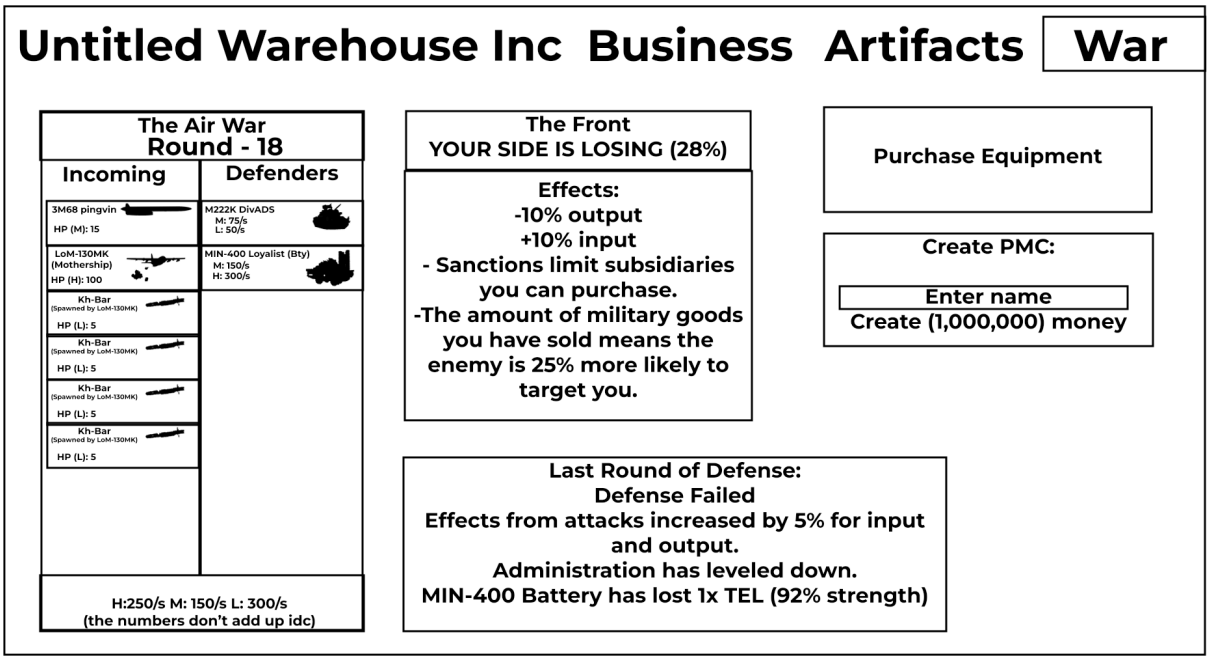
handle_front() will generate the blurb for the font. War score is stored as a int in state and handle_front() will modify the other attributes in State shown in the picture below.

The air war section will be a mysterious piece (or several pieces of code) that an LLM will generate and which I will spend most of my time analyzing. It might work of a list of things attacking and things defending with a function calculating a score for each side from looping through item in the list and summing the combat power.

handle_equipment() will handle the purchase equipment tab and modify a list in State with each value as a string that represents what the player has.

handle_war_round() will change parameters in State or a dataclass in State called War that relate to debuffs applied to the player. Things like damage to equipment are stretch goals and will not be elaborated on.

Create PMC is a stretch goal and will not be elaborated on.



2) Implementing Requirements

This table should serve as a draft for how you will implement the requirements into your final project. You may change your ideas later on, but your final site must fulfill all the requirements.

Requirement	**Implementation**
Purpose/Functionality A 1-2 sentence description of the purpose or functionality of your site	**The site provides entertainment in the form of an easy-to-pick-up game that runs completely in the browser.**

State Dataclass Attributes Minimum of 4 attributes, one of which must be a list/dict of instances of your second dataclass. Briefly describe each attribute, its type, and how it will be meaningfully modified throughout the site.	1. goods_stocked: Int. amount of goods the player has stocked. Will be modified by looping functions that take goods_stock_per_second and goods_sold_per_second and change goods_stocked. 2. goods_stocked_per_second: Int. amount of goods the player stocks per second. Modified by a looping function that take various factors, like the amount of procurement specialists the player has, and return an appropriate value that reflects the player's status. 3. goods_sold_per_second: Int. amount of goods the player sells per second. Modified by a looping function that takes various factors, like the amount of marketers the player has, and returns an appropriate value that reflects the player's status. 4. obtained_artifacts: list[Artifacts] A list of artifacts the player has obtained. Some function (probably related to expedition) will append a artifact from a big list somewhere else over to this parameter when the player is supposed to receive a new artifact.
Second Dataclass Your second dataclass, including its attributes, their types, and how they will be meaningfully modified (if they are modified)	Artifacts: has_obtained: bool. Starts at false and may be changed by the function(s) that handles expeditions. Represents the artifacts the player has so it will inevitably change. goods_bonus: int. assigned to a artifact instance at the start, does not change. admin_bonus: int. assigned to an artifact instance at the start and does not change. Other bonuses following the same structure above.
Routes (minimum 7)	1. index()

The names of your route functions	2. x5_adjust() returns index 3. increase_markup() returns index 4. lower_markup() returns index 5. business() 6. war() 7. artifacts()
Pages (minimum 5) A brief description of the pages in your site	1. Index: main page where the player buys material and sees the stats and function of their business 2. Business: page where the player can select what goods they want to sell and access other business related pages 3. Artifacts: page where the player can see their artifacts, send expeditions, and organize their artifacts for bonuses. 4. War: page where the player can handle war affairs 5. Poker: page where the player can play poker for in-game-currency Alternatively index becomes a page for the player to name their company, and the main page then becomes “company.”
User Input (minimum 3)	1 Textbox: Name company at the start on index page.

Description of user input components (TextBox, TextArea, CheckBox, or SelectBox) and their purposes For more information on these input types, see Drafter Documentation.	The name is displayed at the top left corner through the game. 2 Checkbox: select policies for the company to implement. Policies change how profitable the company is. Checkboxes are a good way of doing that. 3 Checkbox: Select goods to sell. Checkboxes are a good way of organizing this.
If Statements (minimum 3) Brief description of what will be evaluated in your if statements and how they will meaningfully change the logic of the site	1. A long if/elif/else statement in handle_purchase_option() will be used to actually determine what handle_purchase_option() returns. What the player has available to them (stored in a dataclass or a list inside of State) will be checked by the if statement in handle_purchase_option. This makes it so that the player will progress in a specific way, dictated by the designer, rather than in any way. 2. A long if/elif/else statement in handle_goods_sold() will be used to determine the final goods bonus of all the items the player decides to sell. The if statement will ensure that the final goods bonus is actually consistent and reflects what items the player sells. 3. handle_front() if statement will assign penalties to the player based on the war situation. It will evaluate an integer from 0-200 representing the war situation and return appropriate penalties to the player.
Loop (minimum 1) This loop must iterate over the list/dict of dataclass instances in your State attribute. Description of this loop and why it is meaningful to the site	There will be a loop to handle the list of artifacts the player should have. It will iterate through the master list of all possible artifacts and find the ones where has_obtained is truthy, appending them to a new list which then replaces whatever list is in State.obtained_artifacts. It is meaningful to the site because the artifact mechanic would not work without it.

LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

Prompt:

Using the drafter library on python and claude code with claude opus, I want to make a tycoon game where you run a warehouse/ecommerce business in the middle of a war. The player has to buy material for their business (procurement agents to buy goods, drivers and trucks to deliver goods, marketers to sell goods, etc), there are things for the player to research to improve their business in a "one time" kind of way, and policies the player can select and also some kind of upgrade that makes the game run faster (it will have a capped speed at first, maybe 1/3 tick per second for example). There will also be a gambling mechanic which contains nonsense games of chance for the player's amusement. There also needs to be a gacha mechanic where players can pay for expeditions to discover artifacts, which they can use to provide bounses to their company (artifacts activate when they are placed into a 6x2 grid and there are adjacency bonuses). Finally, there is a war mechanic where, after selling enough "war supplies" which provide the player with massive income bonuses, the player's warehouses face attacks, and the player has to buy military equipment to defend their warhouses. How viable is it to vibecode this over the course of a week? [sic]

Response:

Honestly? **Very viable if you're willing to let Claude write 80-90% of the code and you're disciplined about scope.**

The version you described is not really "a game." It's more like **five interconnected idle games**:

1. Core warehouse/ecommerce economy
2. Research tree
3. Artifact/gacha system
4. Gambling minigames
5. War/defense layer

The good news is that none of these require advanced algorithms, graphics, networking, or difficult programming. They're mostly:

- numbers
- state management
- buttons
- timers

- formulas

which is exactly the sort of thing that LLMs are surprisingly good at generating.

What I think is possible in 1 week

Definitely feasible

Core economy:

- Money
- Goods inventory
- Procurement agents
- Marketers
- Drivers
- Trucks
- Warehouse capacity
- Tick-based production

Research:

- Buy once
- Permanent bonuses
- Research trees

Policies:

- Choose one of several policies
- Bonuses and penalties

Speed upgrades:

- Tick speed increases
- Maximum cap

Gambling:

- Slots
- Coin flip
- Roulette-esque nonsense

Simple war mechanic:

- Enemy strength value
- Warehouse defense value
- Periodic attacks

I'd expect Claude to write all of this fairly quickly.

Feasible but could eat time

Artifact system.

The 6x2 artifact grid is actually its own game.

You need:

[A][ ][B][ ][ ][ ]

[C][D][ ][ ][ ][ ]

Each artifact needs:

- rarity
- effects
- adjacency bonuses
- placement/removal
- expedition acquisition

That alone could become 1,000+ lines if done properly.

But Claude should still be able to build it.

Biggest risk: UI

The challenge is not the mechanics.

The challenge is:

How do I display all these systems?

Drafter is not Unity.

Drafter is basically:

```
Page(state, [
        Button(...),
```

Text(...),

])

You will spend more time organizing menus than implementing formulas.

For example:

Main Dashboard

Research

Procurement

Sales

War

Artifacts

Casino

Probably each becomes its own page.

What I'd do if I were vibecoding it

Day 1

Economy

Get this working:

Money

Goods

Buy agents

Buy marketers

Buy trucks

Tick system

Nothing else.

Day 2

Research

Add:

+10% revenue

+20% procurement

+25% warehouse capacity

etc.

Day 3

War supplies

Introduce:

Regular Goods

War Supplies

War supplies become the prestige-like route.

Day 4

War mechanic

Add:

Threat level

Attacks

Defenses

Armor

AA guns

Bunkers

Keep it simple.

Day 5

Artifact system

Get:

12 slots

artifact bonuses

expedition button

working.

Ignore adjacency initially.

Day 6

Adjacency bonuses.

This is where your time goes.

Day 7

Add casino nonsense.

Since it's isolated from everything else, it's easy to bolt on at the end.

The real danger

Feature creep.

You already mentioned:

- economy
- war
- research
- policies
- speed upgrades
- gambling
- artifacts
- adjacency bonuses
- expeditions

That's enough.

What usually happens is around day 3 you think:

> "Wouldn't it be cool if there were multiple warehouse regions?"

Then:

> "What about supply routes?"

Then:

> "What about diplomacy?"

Then:

> "What about tech levels?"

And suddenly your 1-week project is trying to become Factorio + Cookie Clicker + Rimworld.

My estimate

With:

- Drafter
- Claude Opus
- Full vibecoding
- ~2-4 hours/day

I'd give the odds roughly:

Outcome	Chance
Basic playable prototype	95%
All major systems implemented	80%
Artifact adjacency system working well	60%
Balanced and fun game	20%
Polished enough that strangers would play it for hours	5%

The coding is the easy part. The balancing is the hard part. Claude can generate 10,000 lines of tycoon code in a week, but getting the progression curve to feel good usually takes much longer than building the systems themselves.

Final Submission

When you are finished, download this file as a Word Document (docx) and upload the file to the appropriate assignment on Canvas.